KHOA ĐIỆN TỬ - TRÍ TUỆ NHÂN TẠO

CUỘN VÀ DANH SÁCH TRONG FLUTTER

GVHD: TS. Nguyễn Duy Nhật Viễn
SVTH: Nguyễn Minh Tuấn
Đoàn Thanh Tuyển

Đà Nẵng, 11/2026

PHÂN CÔNG CÔNG VIỆC

TT	Tên	Nội dung	Đóng góp
1	Đoàn Thanh Tuyển	SingleChildScrollView, ListView, ListView.builder, GridView. Chuẩn bị code minh họa.	50%
2	Nguyễn Minh Tuấn	Cuộn lồng nhau, lazy loading, dữ liệu dài, trạng thái rỗng. Chuẩn bị demo và slide.	50%

I. CƠ SỞ LÝ THUYẾT

CÁC WIDGET CUỘN TRONG FLUTTER

Có 4 kiểu Widget cuộn trong flutter:

- **SingleChildScrollView:** cuộn một widget con duy nhất, phù hợp nội dung ít và cố định.
- **ListView:** danh sách cuộn một chiều, children viết sẵn.
- **ListView.builder:** tạo item theo nhu cầu, phù hợp danh sách dài hoặc động.
- **GridView:** lưới nhiều cột, có bản builder cho dữ liệu lớn.

SINGLECHILDSCROLLVIEW

- Bọc một widget (thường là Column hoặc) để cuộn khi nội dung vượt màn hình.
- Build toàn bộ con ngay lập tức, không lazy.
- Dùng cho form, trang chi tiết.
- Không có cơ chế tái sử dụng

```
SingleChildScrollView(
  padding: EdgeInsets.all(16),
  child: Column(
    children: [
      Text('Tiêu đề'),
      Text('Nội dung dài...'),
      ElevatedButton(
        onPressed: () {},
        child: Text('Gửi')),
    ],
  ),
)
```

LISTVIEW

Widget hiển thị một danh sách các phần tử theo dạng tuyến tính (dọc hoặc ngang).

- Nhận danh sách **children** cố định.
- Mặc định cuộn dọc, đổi bằng scrollDirection.
- Phù hợp danh sách ngắn (menu, cài đặt).

```
ListView(
  children: [
    ListTile(title: Text('Hồ sơ')),
    ListTile(title: Text('Cài đặt')),
    ListTile(title: Text('Đăng xuất')),
  ],
)

// ListView.separated: thêm đường kẻ
// giữa các item
```

LISTVIEW.BUILDER

Là contrucrtor tối ưu của Listview dùng cho các danh sách dài hoặc danh sách động có số lượng phần tử lớn

- Chỉ build item đang hiển thị (lazy).
- Cần **itemCount** và **itemBuilder**.
- Phù hợp danh sách dài hoặc dữ liệu động.

```
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, i) {
    return ListTile(
      title: Text(items[i]),
    );
  },
)
```

GRIDVIEW

Tương tự như Listview nhưng sắp xếp các phần tử theo dạng lưới 2 chiều

- **GridView.count:** số cột cố định.
- **GridView.extent:** giới hạn chiều rộng ô.
- **GridView.builder:** lazy cho dữ liệu lớn.

```
GridView.builder(
  gridDelegate:
    SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: 2,
      mainAxisSpacing: 8,
      crossAxisSpacing: 8,
    ),
  itemCount: items.length,
  itemBuilder: (c, i) =>
    Card(child: Text(items[i])),
)
```

CUỘN LỒNG NHAU

Các cách xử lý hiện tượng cuộn lồng nhau (Nested Scrolling) trong Flutter

shrinkWrap: true

Cho list co theo nội dung con, nhưng build toàn bộ item nên mất lazy.

physics

NeverScrollableScrollPhysics() tắt cuộn của list con, để list cha cuộn.

Expanded

Khi đặt list trong Column, bọc Expanded để list có chiều cao xác định.

CUSTOMSCROLLVIEW VÀ SLIVER

- Khái niệm: CustomScrollView là một widget cuộn bậc cao cho phép bạn tự kết hợp nhiều vùng cuộn khác nhau vào chung một trục cuộn duy nhất.

- Bản chất: Nếu ListView chỉ ép bạn hiển thị một danh sách đồng nhất, thì CustomScrollView hoạt động như một "vỏ bọc" quản lý một danh sách các Sliver bên trong nó. Mọi chuyển động kéo/cuộn của người dùng sẽ được phân phối đồng thời xuống tất cả các Sliver này.

```
CustomScrollView(
  slivers: [
    SliverToBoxAdapter(child: Header()),
    SliverList.builder(
      itemCount: items.length,
      itemBuilder: (c, i) =>
        ListTile(title: Text(items[i])),
    ),
  ],
)
```

CUSTOMSCROLLVIEW VÀ SLIVER

- Khái niệm: "Sliver" (có nghĩa là một mảnh/thanh nhỏ) là các khối xây dựng cơ bản bên trong CustomScrollView.

- Cơ chế hoạt động:

    - Mọi widget cuộn thông thường trong Flutter (như ListView, GridView) thực chất bên dưới đều được bọc bởi các Sliver tương ứng (SliverList, SliverGrid).
    - Các Sliver tuân theo một giao thức layout chuyên biệt gọi là SliverConstraints và SliverGeometry.

```
CustomScrollView(
  slivers: [
    SliverAppBar(
      expandedHeight: 200.0,
      pinned: true,
      flexibleSpace: FlexibleSpaceBar(title: Text('Tiêu đề co giãn')),
    ),
    SliverList(
      delegate: SliverChildBuilderDelegate(
        (context, index) => ListTile(title: Text('Item $index')),
        childCount: 10,
      ),
    ),
  ],
)
```

LAZY LOADING VÀ HIỆU NĂNG

- Lazy ở mức giao diện: chỉ tạo item trong vùng nhìn thấy (cộng cacheExtent).
- Dùng **const** widget và key ổn định cho item.
- Đặt itemExtent hoặc prototypeItem để bỏ bước đo kích thước.
- Tránh Column + map() trong SingleChildScrollView với danh sách dài.
- Ảnh lớn: dùng cacheWidth/cacheHeight để giảm bộ nhớ.
- Đo bằng Flutter DevTools → Performance (profile mode).

XỬ LÝ TẬP DỮ LIỆU DÀI

- Lazy Loading (ListView.builder)
  - Chỉ render các item nằm trong vùng nhìn thấy (Viewport).
  - Tiết kiệm tối đa bộ nhớ RAM khi hiển thị hàng ngàn phần tử.
- Phân trang (Pagination / Infinite Scroll)
  - Chia dữ liệu thành từng trang nhỏ thay vì tải tất cả một lúc.
  - Tự động gọi API tải thêm (Load More) khi cuộn đến cuối danh sách.
- Tối ưu hóa Rebuild
  - Sử dụng từ khóa const cho các widget tĩnh bên trong item.
  - Tách nhỏ state của từng item, tránh gọi setState ở widget cha gây giật lag toàn bộ danh sách.

```
_ctrl.addListener(() {
  final p = _ctrl.position;
  if (p.pixels >= p.maxScrollExtent - 200
      && !isLoading && hasMore) {
    loadMore();
  }
});

itemCount: items.length + (hasMore ? 1 : 0),
itemBuilder: (c, i) => i == items.length
  ? CircularProgressIndicator()
  : ItemTile(items[i]),
```

TRẠNG THÁI DANH SÁCH RỖNG

- Tránh giao diện trống trơn khó hiểu: Nếu không xử lý, màn hình sẽ hiển thị một khoảng trắng hoặc một ListView rỗng, khiến người dùng bối rối không biết ứng dụng đang bị lỗi hay thực sự không có dữ liệu.

- Định hướng hành động cho người dùng (Call to Action): Cung cấp thêm thông tin giải thích và một nút bấm hành động (như "Thêm mới", "Tải lại", "Quay lại trang chủ") giúp người dùng biết họ nên làm gì tiếp theo.

```
if (items.isEmpty) {
  return Center(
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Icon(Icons.inbox, size: 64),
        Text('Chưa có dữ liệu'),
        ElevatedButton(
          onPressed: reload,
          child: Text('Tải lại')),
      ],
    ),
  );
}
return ListView.builder(...);
```

II. THỰC HÀNH VÀ DEMO

DEMO 1: SINGLECHILDSCROLLVIEW

- Bọc mộtColumn trong SingleChildScrollView để cuộn khi nội dung vượt màn hình.
- Dựng toàn bộ 12 thẻ cùng lúc, không lazy.
- Phù hợp nội dung ít và cố định: form, trang chi tiết.




```
SingleChildScrollView(
 padding: const EdgeInsets.all(16),
child: Column(children: [
        Text('Thông tin sinh viên'),
        for (int i = 1; i <= 12; i++) Card(...),
 ]),
)
```

DEMO 2: LISTVIEW.BUILDER VÀ LAZY LOADING

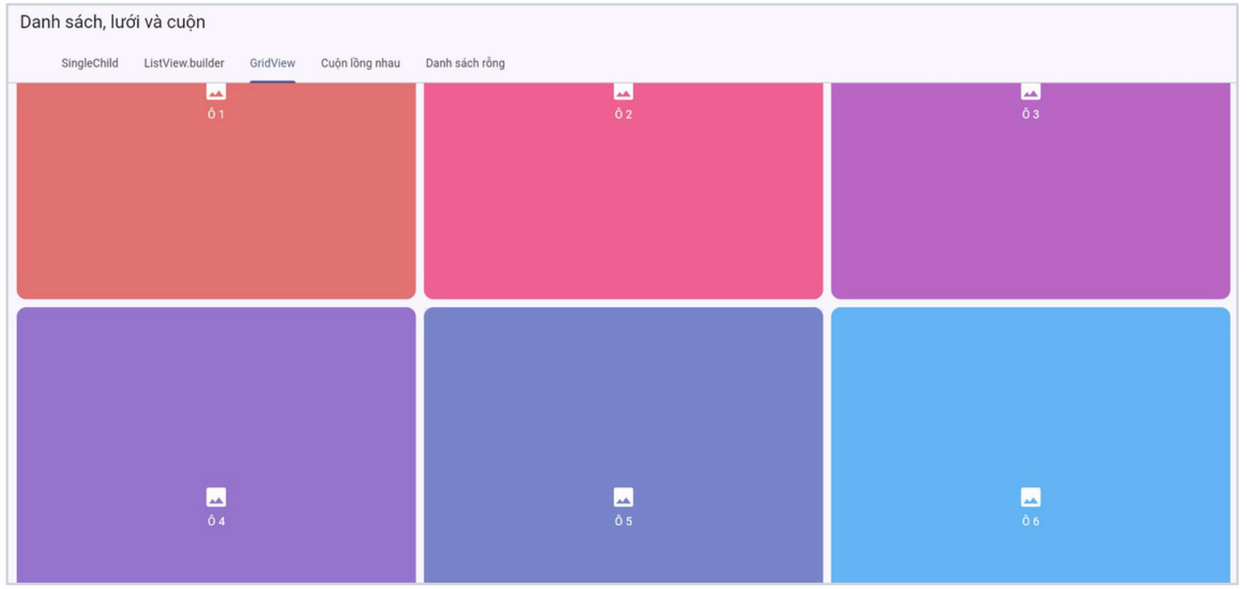
- **Dataflow:** cuộn → _onScroll() → _loadMore() → setState → builder dựng lại.
- Còn cách đáy 200px thì tải thêm 20 item.
- Cờ _isLoading chặn tải trùng khi cuộn nhanh.
- Item cao thấp khác nhau do nội dung dài ngắn khác nhau.




```
_controller.addListener(_onScroll);
itemCount:  _items.length  +  (_isLoading  ?  1  :  0),
itemBuilder: (context, index) {
 if (index == _items.length) {
       return const CircularProgressIndicator();
   }
 ...
```

DEMO 3: GRIDVIEW.BUILDER

- Lưới 3cột,60ô, mỗi ô được tạotheo nhu cầu (lazy).
- **crossAxisCount: 3** nên mỗi hàng có 3 ô. **childAspectRatio** quyết định tỉ lệ rộng / cao của ô.
- Trên màn hình rộng, ô to ra theo chiều ngang. Muốn cố định kích thước ô, dùng MaxCrossAxisExtent.




```
GridView.builder(
itemCount: 60,
gridDelegate:
          const SliverGridDelegateWithFixedCrossAxisCount(
       crossAxisCount: 3),
 itemBuilder: (context, index) => ...,
)
```

DEMO 4: CUỘN LỒNG NHAU

- Danh sách cuộn ngang(Thẻ1 đến 15) nằm trên, danh sách cuộn dọc nằm dưới, cùng trong một SingleChildScrollView.
- **SizedBox(height: 130):** list ngang cần chiều cao cố định.
- **shrinkWrap + NeverScrollableScrollPhysics:** để list cha xử lý cuộn.




```
SizedBox(
 height: 130, // list ngang
child: ListView.builder(
        scrollDirection: Axis.horizontal, ...),
),
ListView.builder(
 shrinkWrap: true, // list dọc co theo nội dung
physics: const NeverScrollableScrollPhysics(), ...),
```

DEMO 5: TRẠNG THÁI DANH SÁCH RỖNG

- **Dataflow:**gõchữ → onChanged → setState(_query) → build() lọc → filtered.isEmpty?
- Gõ F: còn 1 kết quả (Flutter).
- Gõ xyz: hiện icon và dòng Không tìm thấy kết quả.
- Chỉ _query là State, filtered được tính lại mỗi lần build.



Gõ F: có kết quả

```
final filtered = _all.where(...).toList();

return filtered.isEmpty
        ? const Center(
                child: Text('Không tìm thấyG kêGt qual'))
        : ListView.separated(
                itemCount: filtered.length, ...);
```